

Lecture 5 — 13/08

Lecturer: Arjun Bhagoji

Scribe: Aditya Agrawal

5.1 Input Representations

We start off by looking at whether the way we represent data for classification or anomaly detection good enough?

A *feature map* $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ is simply a function which maps the input data into another space known as the *feature* or *embedding* space.

In k-class classification problems, we need a *map* $f : X \rightarrow [0, 1]^k$ to denote *confidence* of a particular input lying in some class.

The standard classification algorithms such as logistic regression, or support vector machines directly operate on the input data while *representational learning* first maps raw data into *feature space* on which standard algorithms are applied. Representational learning is generally of two types:

- (a) *Fixed* representations: These are explicit maps which are used on basis on some prior knowledge of data distribution. For example, if in a text classification task, we had a dictionary of semantic meanings of words, our feature map could explicitly map them to the same representation
- (b) *Learned* representations: These are maps which are *learnt* through Neural Networks, or other such techniques

A *kernel* is defined as a dot product in feature space:

$$k : X \times X \rightarrow \mathbb{R} \quad k(x, x') = \langle \phi(x), \phi(x') \rangle$$

The standard dot product arises when we use the identity feature map $\phi(x) = x$.

5.1.1 Feature maps from kernels

We ask the question of when do arbitrary kernel functions arise as a dot product between some feature vectors.

Theorem 5.1. *If $k(x, x')$ is real valued and positive semi definite, then there exists an inner product space $\mathcal{X}$ and feature map $\phi : \mathbb{R}^d \rightarrow \mathcal{X}$ such that $k(x, x') = \langle \phi(x), \phi(x') \rangle$*

We simply define $\mathcal{X}$ as the set of all maps $f : \mathbb{R}^d \rightarrow \mathbb{R}$ with an appropriate inner product and the feature map $\phi : \mathbb{R}^d \rightarrow \mathcal{X}$ as $\phi(x)(\cdot) = k(x, \cdot)$. We also have to validate the existence of the inner product on the function space but the positive semi definiteness of the product will allow us to do so.

5.1.2 Examples of Kernels

- (a) Polynomial Kernels : $k(x, x') = \langle x, x' \rangle^d$
- (b) Gaussian Kernels : $k(x, x') = e^{-\|x - x'\|^2 / 2\sigma^2}$

5.1.3 The Kernel Trick

Well, the representations we dealt with earlier either fixed or learned were into finite dimensional spaces. But fortunately, in all the dual formulations of our optimization equations, the mentions of $\phi(x)$ were all as dot products of the form $\phi(x)^T \phi(x')$, which can instead be replaced by $\tilde{k}(x, x')$ leading to a feature map $\tilde{\phi} : \tilde{\phi}(x)(\cdot) = \tilde{k}(x, \cdot)$ where this feature map could be infinite dimensional!!

5.2 Anomaly Detection

We come back to the original question of anomaly detection and propose another method.

5.2.1 One Class SVM

Consider the following constrained parametric optimization problem:

$$\begin{aligned} \min_{w, \gamma, \rho} \quad & \frac{1}{2} \|w\|^2 + \frac{1}{vn} \left(\sum_{i=1}^n \gamma_i \right) - \rho \\ \text{subject to} \quad & \begin{cases} \langle w, \phi(x_i) \rangle \geq \rho - \gamma_i & i = 1, \dots, n, \\ \gamma_i \geq 0 & i = 1, \dots, n, \end{cases} \end{aligned}$$

where v is a hyperparameter which asymptotically represents the fraction of anomalies. ρ represents the margin of confidence we desire, higher the ρ , the further away an in distribution point is required to be away from the classification boundary.

At test time, we define a scoring map $s_\theta(x)$ and a hypothesis map $h_\theta(x)$ as the following:

$$\begin{aligned} s_\theta(x) &= \langle w, \phi(x) \rangle - \rho \\ h_\theta(x) &= \text{sgn}(f_\theta(x)) \end{aligned}$$

Note: The scoring map indicates how likely our given point is in distribution. A high score indicates being in distribution while a negative score likely reflects an anomaly. Finally, the map h is the one which maps the input into a class 0 or 1 where 0 indicates an anomaly.

5.2.2 HW Problems

1. Derive the primal optimization problem of the *lagrangian* of a standard one class SVM machine for both the soft and the hard cases using the KKT conditions
2. Derive the same for the v -SVM as well
3. Derive the dual of these primal objective functions as well

We provide the solution to the one of the *lagrangian* objective function mentioned above, the rest of them are left as exercises to the student, by using the KKT conditions.

We are given:

$$\begin{aligned} \langle w, \phi(x_i) \rangle &\geq \rho - \gamma_i & i = 1, \dots, n, \\ \gamma_i &\geq 0 & i = 1, \dots, n, \end{aligned}$$

We use the set of variables α_i to model $\rho - \gamma_i - w^T \phi(x_i) \leq 0$, the set η_i to model $-\gamma_i \leq 0$ and μ to model $-\rho \leq 0$. The lagrangian we then get is:

$$\begin{aligned} \mathcal{L}(w, b, \rho, \gamma, \alpha, \eta, \mu) = & \frac{1}{2} \|w\|^2 - \rho + \frac{1}{vn} \sum_{i=1}^n \gamma_i + \sum_{i=1}^n \alpha_i \left(\rho - \gamma_i - (w^\top \phi(x_i) + b) \right) - \sum_{i=1}^n \eta_i \gamma_i - \mu \rho \\ & \alpha_i \geq 0, \eta_i \geq 0, \mu \geq 0 \end{aligned}$$

On applying the KKT conditions to these, we get the dual objective function as:

$$\begin{aligned} \max_{\alpha} \quad & -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \phi(x_i)^T \phi(x_j) \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq \frac{1}{vn}, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \alpha_i \geq 1 \quad (\text{and } \sum_{i=1}^n \alpha_i = 1 \text{ when } \rho > 0). \end{aligned}$$

5.2.3 SLT- like Guarantees on the One Class SVM

Theorem 5.2. (informally [10]) Let $R_{w,\gamma} = \{x | f_\theta(x) \geq 0\}$. Then for all $\delta > 0$, with probability $1 - \delta$ over the draws $\{x_i\}_{i=1}^n$, for any $B > 0$, we get

$$\mathbb{P}_{x \sim \mathbb{P}^+} [x | x \notin R_{w,\rho-B}] \leq \frac{2}{n} \left(k(n, B, \|W\|, \rho) + \log_2 \left(\frac{n^2}{2\delta} \right) \right)$$

where k is some non-negative function of problem parameters

5.2.4 A Unifying View of Anomaly Detection

In essence, anomaly detection involves designing *score functions* which work well during test time. When the score of a particular sample is high enough, we deem it to be from our sample distribution. Students can check out [7]

5.3 Modern OOD Detection

The OOD problem or the *Out Of Distribution* problem is the semi supervised case of the anomaly detection problem where we are now also provided labels on the in distribution samples. Our key question is to understand how OOD detection is done nowadays. Please check out [9] for a comprehensive review on OOD detection.

5.3.1 Deep OCSVM or Deep SVDD

In this methodology ([2],[8]) , we use a learned feature map $f_\theta(x)$, where θ paramterizes some neural network, with θ learned is jointly over the supervised learning component and the anomaly detection (R, c) or (w, γ, ρ) . The anomaly detection optimization problem is given as

$$\min_{R, c, \theta} R^2 + \frac{1}{vn} \sum_{i=1}^n \max(0, \|f_\theta(x_i) - c\|^2 - R^2) + \text{Reg}(\theta)$$

Here, typically some regularization is put on θ since otherwise it tends to fit maps such that all points in feature space start falling within the sphere. The neural network is typically trained through stochastic gradient descent.

5.3.2 MSP and ODIN ([3],[6])

MSP stands for *Maximum Softmax Probability* and works on the principle that a neural network holds enough discriminative power to classify in distribution samples confidently. This leads to the formulation where we call an input out of distribution when the neural network cannot classify it confidently. We can check the score of the most probable class as assessed by the neural network to this end. The neural network can be parametrized by a function $f_\theta(x)$ which outputs logits corresponding to the confidence of being in that particular class

ODIN or *Out of Distribution for Neural Networks* enhances MSP by preprocessing data smartly and adding a temperature control on the softmax performed. It works in the following way:

1. Preprocesses input x as

$$x' = x + \varepsilon * \text{sgn} \left(\nabla_x \log(f_{\hat{y}}^\theta(x, \tau)) \right)$$

$$f_\theta^i(x, \tau) = \frac{e^{\frac{l_i(x)}{\tau}}}{\sum_{i=1}^n e^{\frac{l_i(x)}{\tau}}}$$

where $l_i(x)$ is the logit output of neural network $l : \mathbb{R}^d \rightarrow \mathbb{R}^p$ and $\hat{y}$ denotes the class with the highest softmax score

2. Defines the score map s_θ and anomaly hypothesis map h_θ as

$$s_\theta(x) = \max_i f_\theta^i(x, \tau)$$

$$h_\theta(x) = 2 * (\mathbf{1}[s_\theta(x) < \delta] - 1) - 1$$

5.3.3 Outlier Exposure [4]

Well the issue with our current methods is that they are overfitted on in distribution data. It is very likely that out of distribution data looks very different and might lead the overfit classifier to falsely label them as being in distribution. To prevent this, we perform outlier exposure by slightly tweaking our NN loss functions. Our goal is to now minimize

$$\min_{\theta} \mathbb{E}_{x \sim \mathbb{P}^+} l(h_\theta(x), y) + \lambda \mathbb{E}_{x \sim \mathbb{P}^-} l_{OE}(f_\theta(x))$$

Here λ controls the rate of outliers we expect in testing and l_{OE} is the loss we want to calculate over outlier data. We generally assume that the out of distribution is uniform in some sense. The OOD data used while training Outlier Exposure does not actually have to match the actual OOD data seen at test time.

5.3.4 Enhancing MSP

We essentially want to temper the confidence of our classifier, by saying "Hey! Don't be so confident about your predictions, they might be on out of distribution data!!". We design two outlier two loss functions for the same.

1.

$$l_{OE}(x) = - \sum_{j=1}^{|C|} \frac{1}{|C|} \log(f_{\theta}^j(x))$$

We *assume* that a good classification is a confident classification, in the sense that if our classifier classified something well, then we inherently assumed such a point belonged to our distribution. We penalize OOD samples for deviation from the uniform distribution

2. To make a distinction between a *good* and *confident* classification we artificially introduce a confidence estimation branch $c(x) \in [0, 1]$ and set $l_{OE}(x) = -\log(c(x))$ [1].

During training, we then try to minimize

$$l(x) = - \sum_{j=1}^{|C|} \log(cf_{\theta}(x_j) + (1-c)y_j)y_j - B\log(c(x))$$

Essentially, what we try to say is that if we are not confident about whether the input is from distribution, then our NN doesn't pay a training loss, however we pay a confidence loss. If we are confident, then we incur a training loss as well. The confidence branch provides a score which can also be used to classify OOD inputs

5.3.5 Grad Norm KL Divergence

Instead of looking at the maximum softmax probability as a measure of confidence, we look at the gradient of the norm of the KL divergence as such a measure. This is the methodology followed in [5]. The intuition is that low gradient norm implies points close to margins and so more likely to be OOD. We define this using

$$\begin{aligned} D_{KL}(U||f_{\theta}(x)) &= \sum_{j=1}^{|C|} \frac{1}{|C|} \log\left(\frac{1/|C|}{f_{\theta}^j(x)}\right) \\ &= -\frac{1}{|C|} \sum_{j=1}^{|C|} \log(|C|f_{\theta}^j(x)) \end{aligned}$$

Now,

$$\nabla_{\theta} D_{KL}(U||f_{\theta}(x)) = -\frac{1}{|C|} \sum_{j=1}^{|C|} \frac{\nabla_{\theta} f_{\theta}(x_i)}{f_{\theta}^j(x)}$$

If the point is far away from the margins, then some value of $f_{\theta}(x_i)$, will be small, leading to high gradient norm. Conversely, if the point is close to margin, then the gradient norm will be smaller.

Our hypothesis function h then is $h_{\theta}(x) = 2 * (\mathbf{1}[\|\nabla_{\theta} D_{KL}\|_2 < \delta] - 1) - 1$

5.3.6 Cons of the above methods

While the above methods are very intuitive and make sense, most of their guarantees are empirical and not theoretical. They have also been designed in an adhoc manner with not much theoretical basis as to why they might be good. It might be useful to try and get some SLT like bounds on these like the one we had for the one class SVM.

Bibliography

- [1] Terrance DeVries and Graham W. Taylor. Learning confidence for out-of-distribution detection in neural networks. *arXiv preprint arXiv:1802.04865*, 2018.
- [2] S.M. Erfani, S. Rajasegarar, S. Karunasekera, and C. Leckie. High-dimensional and large-scale anomaly detection using a linear one-class svm with deep learning. *Pattern Recognition*, 58:121–134, 2016.
- [3] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. Poster, OpenReview.
- [4] Dan Hendrycks, Mantas Mazeika, and Thomas G. Dietterich. Deep anomaly detection with outlier exposure. *arXiv preprint arXiv:1812.04606*, 2018. ICLR 2019.
- [5] Rui Huang, Andrew Geng, and Yixuan Li. On the importance of gradients for detecting distributional shifts in the wild. *arXiv preprint arXiv:2110.00218*, 2021. Presented at NeurIPS 2021.
- [6] Shiyu Liang, Yixuan Li, and R. Srikant. Enhancing the reliability of out-of-distribution image detection in neural networks. *arXiv preprint arXiv:1706.02690*, 2017. Presented at ICLR 2018.
- [7] Lukas Ruff, Jacob R. Kauffmann, Robert A. Vandermeulen, Grégoire Montavon, Wojciech Samek, Marius Kloft, Thomas G. Dietterich, and Klaus-Robert Müller. A unifying review of deep and shallow anomaly detection. *arXiv preprint arXiv:2009.11732*, 2020.
- [8] Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib A. Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 4390–4399, 2018.
- [9] Mohammad Salehi, Hossein Mirzaei, Dan Hendrycks, Yixuan Li, Mohammad Hossein Rohban, and Mahdi Sabokrou. A unified survey on anomaly, novelty, open-set, and out-of-distribution detection: Solutions and future challenges. *arXiv preprint arXiv:2110.14051*, 2021.
- [10] Bernhard Schölkopf and Alexander J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 2002.